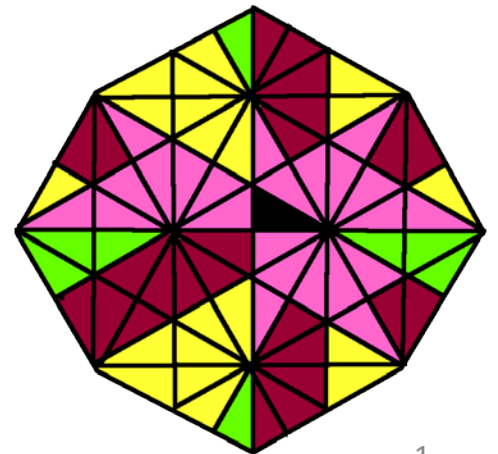


Structuri de Date

Anul universitar 2019-2020

Prof. Adina Magda Florea



Curs Nr. 9_2

Grafuri

- ☐ Exemple
- ☐ Noțiuni de bază
- ☐ Probleme asociate grafurilor
- ☐ ADT Graf
- ☐ Reprezentarea grafurilor
- ☐ Parcurgerea grafurilor
- ☐ **Componente conexe (union-find)**

7. Componente conexe

- Graf neorientat
- Cum putem afla componentele conexe ale unui graf neorientat?
- Componente conexe – vezi **visit**
- Componente conexe – algoritmul Union-Find

Componente conexe

Union-Find

- Este un nod x conectat la un nod y ?
- Grafurile pot fi văzute ca o colecție de obiecte
- **Noduri = obiecte**
- **Arce = “în aceeași mulțime cu”**
- Parcurgere graf, asociem fiecărei componente conexe un indice
- Dar dacă avem un aspect dinamic: vin noi arce apoi întrebăm dacă un nod x conectat la un nod y , etc.



Union-Find

- Algoritm care să răspundă dacă 2 noduri **x** și **y** sunt în **aceeași mulțime** (aceeași componentă conexă) și dacă nu sunt **să le unim** a.î. să fie în aceeași mulțime
- **Find:** x și y sunt în aceeași componentă conexă sau mulțime?
- **Union:** x și y se leagă în aceeași componentă conexă
- Reprezentare specială care permite aceste operații
- Folosim o **pădure de arbori**
- **Find** = sunt x și y în același arbore?
- **Union** = leagă cei 2 arbori în unul singur

Algoritmul Union-Find

- Folosim o reprezentare a arborilor din pădure printr-un vector de părinți **dad[V]**
- Vectorul **dad[i]=j** $\Rightarrow$ **j este nodul părinte a lui i**
- **Obs:** Singura relație între arborii UF și graf este faptul că toate nodurile dintr-un arbore UF sunt în aceeași componentă conexă în graf

Algoritmul Union-Find

```
int find(int x, int y, int doit)
{
    int i=x, j=y;
    while (dad[i]>0) i = dad[i];
    while (dad[j]>0) j = dad[j];
    if ((doit!=0) && (i!=j)) dad[j]=i;
    return(i!=j)
}
```

Algoritmul Union-Find

- Algoritmul are **performanțe proaste** pentru cazul cel mai defavorabil deoarece arborii care se formează pot degenera
- $O(V^2)$ pentru a construi, $O(V)$ pentru a face un test

Algoritmul Union-Find

- Pentru a minimiza distanța la rădăcină, este bine să păstrăm rădăcina arborelui cu cel mai mare număr de descendenți
- **Echilibrarea greutății** (**weight balancing**) = se ține minte numărul de descendenți ai rădăcinii în vectorul **dad[V]** codificat ca un **număr negativ** a.î. să poată fi detectată rădăcina
- **Compresia căii** (**path compression**) = nodurile pe care le parcurgem le facem să **puncteze către rădăcină**
- Forma efectivă a arborelui nu este relevantă
- Algoritm – vezi tabla

Algoritmul Union-Find

```
int find_wp(int x, int y, int doit)
{
    int t, i=x, j=y;
    while (dad[i]>0) i = dad[i];
    while (dad[j]>0) j = dad[j];
    while(dad[x]>0)
    { t = x; x = dad[x]; dad[t] = i; }
    while(dad[y]>0)
    { t = y; y = dad[y]; dad[t] = j; }
    if ((doit!=0) && (i!=j))
    { dad[j] += dad[i]-1; dad[i] = j; }
    else
    { dad[i] += dad[j]-1; dad[j] = i; }
    return(i!=j)
}
```

Algoritmul Union-Find

- Dacă se folosește **weight balancing** și **path compression** atunci numărul total de operații necesare pentru a construi o structură cu **E arce este aproape (dar nu chiar) liniar**.
- Numărul de operații necesare este **$O(E\alpha(E))$**
- $\alpha(E)$ este o funcție care crește foarte încet
- **$\alpha(E) < 4$ atât timp cât**
 - **$\lg(\lg(\dots \lg(E)) < 1$ (de 16 ori)**
- Permite să se determine dacă un graf cu V noduri și E arce este conex în spațiu proporțional cu V și timp aproape liniar
- De ex V - mii de noduri și E - milioane de arce